

Learning Transformations Between Breast-Phantom Volumes with Flow Matching

From a Failed Interpolation Attempt to a Validated 180° Rotation Task

Daniel Kozin

ID 215550260

Diffusion Models, Course Project

Abstract

We train a flow-matching model over a dataset of 3D breast-phantom voxel volumes and use it for two tasks. The first, our original goal, is to interpolate between two real phantoms by inverting each to the model’s noise space, blending there, and integrating forward. This approach produces excellent single-phantom reconstructions but unreliable interpolations: generated volumes either collapse onto one endpoint or tear into disconnected anatomy, a failure that persists across more training, a class-weighted loss, more integration steps, and a partial-inversion variant of the method. We trace this to a structural property of flow matching rather than a bug: with no compression bottleneck, nothing in training constrains the region between two different real phantoms’ noise pre-images. We then pivot to a self-supervised task with exact ground truth, predicting a phantom rotated 180°, which lets us iterate against a known-correct answer instead of visual judgment alone. A sequence of diagnosed failures (an undercommitted velocity field, an over-permissive cross-entropy term, and finally exposure bias between teacher-forced training and the model’s own multi-step rollout) leads to a rollout-consistency loss that raises combined mask Dice from 0.13 to 0.453 ± 0.151 on 176 held-out validation pairs, with no evidence of overfitting.

1 Introduction

Flow-matching and diffusion models learn a transformation from a simple noise distribution to a complex data distribution by regressing a velocity field along straight-line paths between paired noise and data samples. Beyond generation, an appealing use of such a model is *editing*: if the noise space the model has learned is smooth and well-organized, moving along a path between two real data points there should decode into a smooth, meaningful sequence of anatomy in between them, rather than the physically meaningless blur a direct voxel-space crossfade would produce. This project set out to test that idea on a dataset of 3D breast-phantom volumes: invert two real phantoms into the model’s noise space, interpolate, and generate the volume in between. That attempt did not work, for reasons that turn out to be informative about what flow matching does and does not guarantee. We diagnose the failure, then pivot to a task the model *can* be trained and evaluated on with exact ground truth, predicting a 180° rotation of a phantom, and report the sequence of loss and evaluation fixes that took that second task from a model that never moves its prediction at all to one with a validated combined mask Dice of 0.453.

2 Data

Each phantom is a 3D voxel volume ($26 \text{ depth slices} \times 128 \times 128$), with every voxel labeled as one of four classes: background, insert (the overall phantom surface), pillar (holding the lump in place), or lump, the class that matters medically, and the rarest one. After removing double/triple-lump phantoms (out of scope for this single-lump framing), 123 files across 40 case IDs remain, expanded to 984 volumes via $8\times$ in-plane rotation augmentation. Figure 1 shows four examples, and Table 1 gives the measured class balance: the two classes that matter, pillar and lump, together make up barely one percent of every volume. This imbalance shapes several of the design and diagnosis decisions throughout the paper.

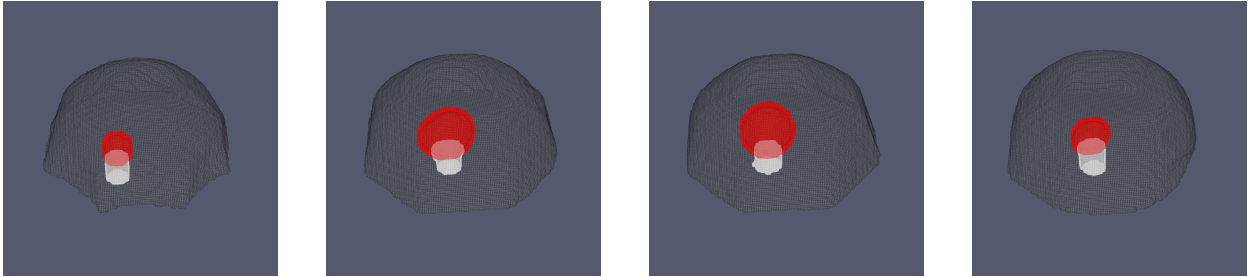


Figure 1: Examples from the breast-phantom dataset. Gray is the phantom surface, red is the lump, and white is the pillar holding the lump.

Class	Background	Insert	Pillar	Lump
Voxel share	50.8%	48.0%	0.32%	0.94%

Table 1: Measured voxel class balance across the dataset.

3 Background: Flow Matching

We have a data distribution we can sample from but have no formula for (real phantoms, one-hot encoded over the 4 classes so each voxel becomes a length-4 vector), and a distribution we understand completely, standard Gaussian noise $x_0 \sim \mathcal{N}(0, I)$ of the same shape. Writing a data sample as x_1 , the simplest path between the two is a straight line,

$$x_t = (1 - t)x_0 + tx_1, \quad t \in [0, 1], \quad (1)$$

whose velocity is constant along the path: $\frac{dx_t}{dt} = x_1 - x_0 =: v^*$. A network $v_\theta(x, t)$ is trained to predict this velocity at randomly sampled points along randomly sampled paths,

$$\mathcal{L}(\theta) = \mathbb{E}_{t \sim U[0, 1], x_0 \sim \mathcal{N}(0, I), x_1 \sim p_{\text{data}}} \left[\|v_\theta(x_t, t) - (x_1 - x_0)\|^2 \right]. \quad (2)$$

Generation integrates this field forward from noise; Euler’s method with n steps of size $\Delta t = 1/n$ repeats

$$x_{t+\Delta t} = x_t + v_\theta(x_t, t) \Delta t, \quad (3)$$

from x_0 at $t = 0$ to a generated $\hat{x}_1$ at $t = 1$; taking the class-wise argmax turns the continuous output back into a discrete label volume. The same field also runs backward: given a real x_1 ,

integrating

$$x_{t-\Delta t} = x_t - v_\theta(x_t, t) \Delta t \quad (4)$$

from $t = 1$ to $t = 0$ recovers an approximate noise pre-image $\hat{x}_0$, i.e. *inversion*.

4 Part I: Interpolating Between Real Phantoms

4.1 Goal and pipeline

To morph between two real phantoms A and B, we invert both to noise ($\hat{x}_0^A, \hat{x}_0^B$), interpolate between the two noise representations at mixing fraction $\alpha \in [0, 1]$, and integrate the result forward through v_θ back to a label volume (Figure 2).

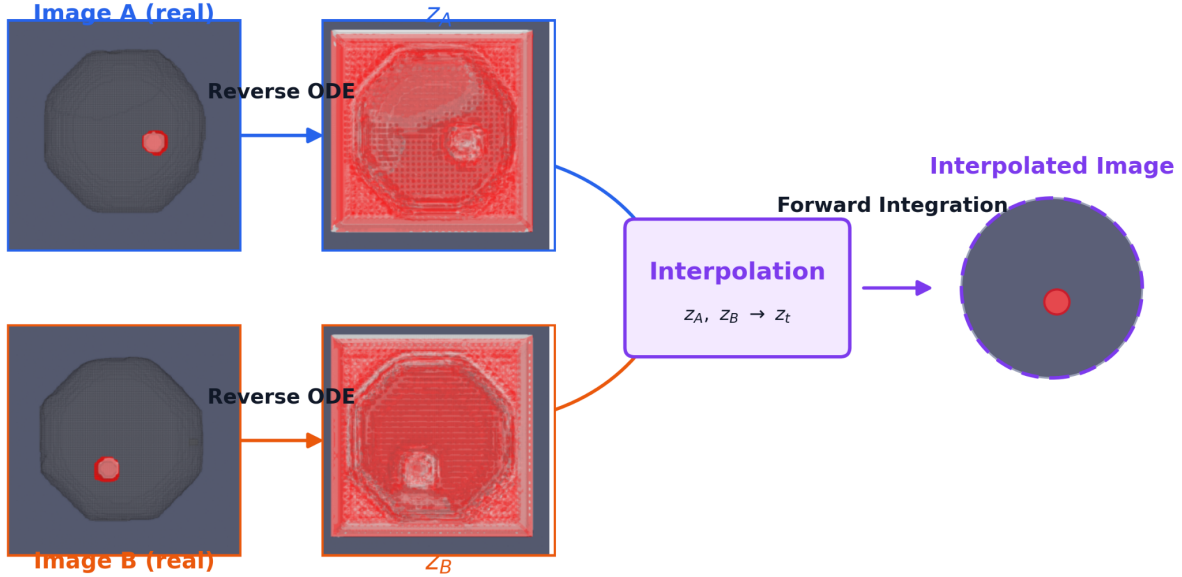


Figure 2: The interpolation pipeline: both real phantoms are inverted to noise, combined, and integrated forward through the learned velocity field.

4.2 Spherical interpolation

A plain linear blend of two Gaussian noise vectors is the wrong operation: in a space of dimension $d \approx 1.7 \times 10^6$ (four channels $\times 26 \times 128 \times 128$), two independent Gaussian samples concentrate tightly around a shell of radius $\sqrt{d}$, while their average has expected norm only $\sqrt{d/2} \approx 0.71\sqrt{d}$, since two independent, essentially orthogonal random directions of equal length shrink when averaged, the same way two arrows in a narrow "V" shrink at their midpoint. This handily quantifies why a linear blend hands the network a point from a region of space it never trained on. Spherical interpolation (slerp) avoids this: it moves along the great-circle arc between the two points instead of stepping through the interior, preserving their common norm exactly at every mixing fraction,

$$\text{slerp}(a, b, \alpha) = \frac{\sin((1-\alpha)\theta)}{\sin\theta} a + \frac{\sin(\alpha\theta)}{\sin\theta} b, \quad \theta = \arccos\left(\frac{a \cdot b}{\|a\| \|b\|}\right). \quad (5)$$

The full pipeline per α is therefore: invert A and B, slerp between $\hat{x}_0^A$ and $\hat{x}_0^B$ at that α , integrate forward, argmax, render.

4.3 Class-weighted loss

Because background and insert dominate the voxel count (Table 1), plain, unweighted MSE (Eq. 2) can be driven close to zero while the model is essentially wrong about the lump. We weight the per-voxel error by class,

$$\mathcal{L}_{\text{weighted}}(\theta) = \mathbb{E} \left[\frac{1}{C \cdot D} \sum_{c=1}^C w_c \sum_{\text{voxels}} (v_\theta - v^*)_{c, \text{voxel}}^2 \right], \quad (6)$$

with weights set to the square root of inverse class frequency (to avoid a handful of rare voxels dominating the gradient), normalized so background=1:

$$w = (1.0, 1.03, 12.57, 7.36) \quad \text{for (background, insert, pillar, lump)}. \quad (7)$$

4.4 Sanity check: inversion round-trip

Before judging interpolation, we checked the model’s ability to invert and reconstruct a *single* real phantom: invert with Eq. 4, then regenerate with Eq. 3 and compare to the original. This gave 96–99% voxel agreement at 50 integration steps across the validation set (98.3% measured specifically on the lump and pillar classes); Figure 3 shows one example. Individual-phantom inversion is therefore excellent — the failure diagnosed below is not a symptom of unreliable inversion.

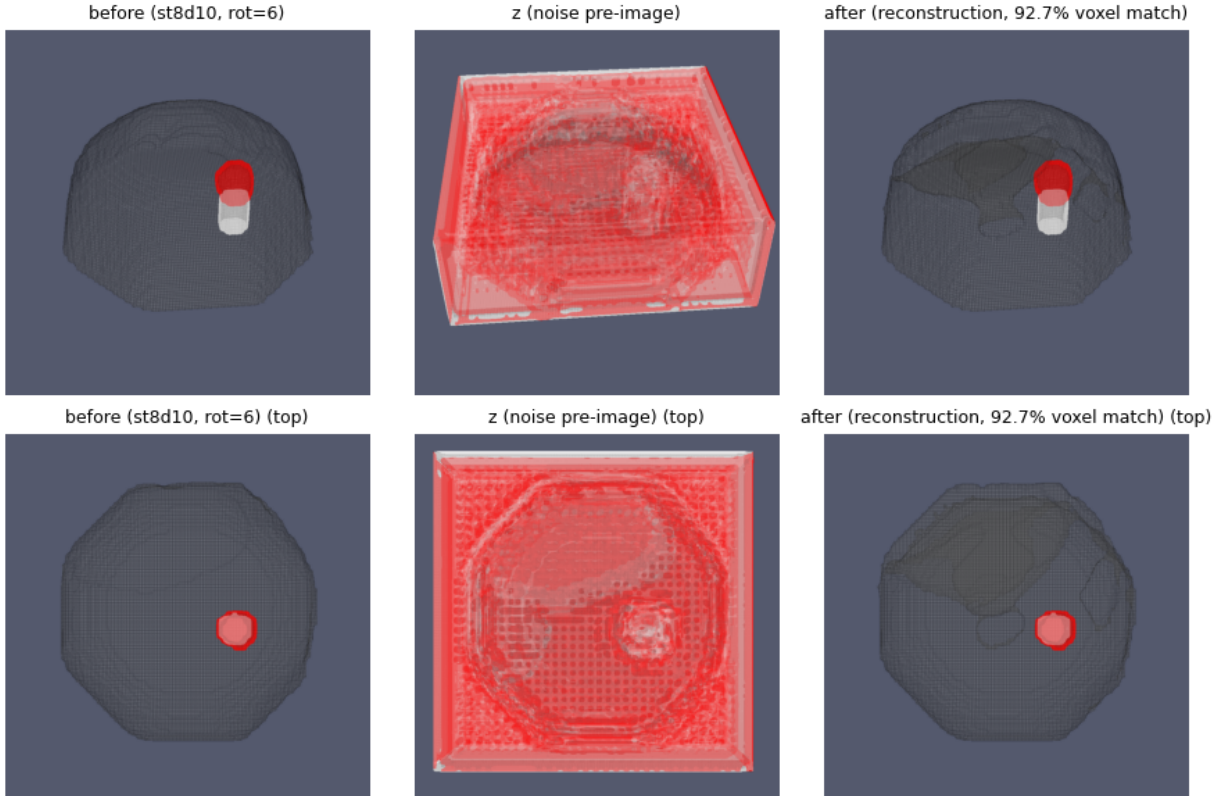


Figure 3: Before (real phantom) $\rightarrow z$ (inverted noise pre-image) $\rightarrow$ after (re-generated from z), default and top-down views. The noise pre-image renders as an oversized, boxy lump region because argmax’d near-Gaussian noise splits roughly evenly across the four channels, unlike real anatomy where background dominates.

4.5 Results: interpolation failure modes

Interpolating between two *different* real phantoms was unreliable. Two failure modes recurred across pairs: the generated volume sometimes collapsed almost entirely onto one endpoint rather than blending; in other cases the predicted lump centroid landed close to the naively expected linear position, yet the surrounding anatomy itself visibly tore, growing extra bumps and disconnected fragments not present in either endpoint (Figure 4). Neither more training epochs, the class-weighted loss of Eq. 6, nor a $3\times$ increase in integration steps changed this pattern, which rules out both undertraining and numerical discretization error as the cause.

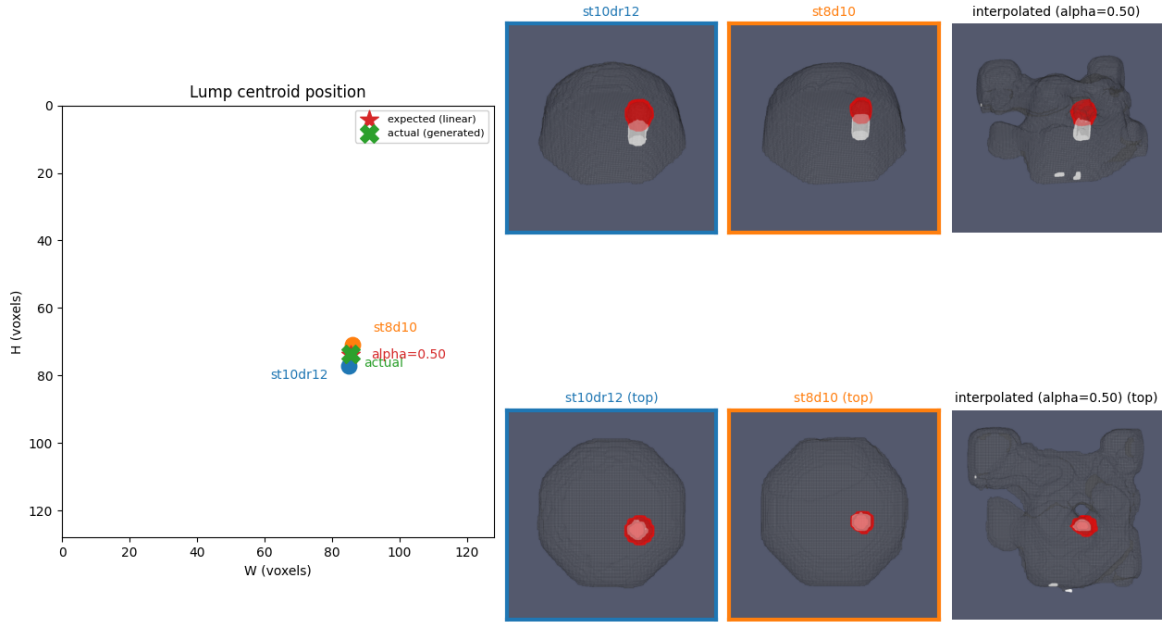


Figure 4: A representative interpolation (pair `st10dr12` ↔ `st8d10`, $\alpha = 0.5$). Left: the naively expected lump position (red star) versus what the model actually generated (green $\times$), close in this case. Right: renders of both endpoints and the interpolated result, showing extra bumps and disconnected specks absent from either endpoint.

4.6 A proposed fix: partial inversion (t_{mid})

Flow matching has no compression bottleneck: the noise space has the same dimension as the data space, and nothing in the training objective (Eq. 2) pushes it to be organized as a whole. Training only ever supervises points that lie exactly on one phantom’s own straight-line path; with only $\sim 10^3$ augmented training volumes scattered across a $\sim 1.7 \times 10^6$ -dimensional space, the neighborhood *between* two different phantoms’ paths, particularly near $t = 0$, the least-constrained region of all, is never touched by training. We tested a direct mitigation: rather than inverting each phantom all the way to $t = 0$, invert only *partway* to an intermediate $t_{\text{mid}} > 0$, interpolate the two partial trajectories there, and integrate forward the remaining distance to $t = 1$ (Figure 5). At larger t_{mid} both trajectories are already pulled closer to their own real anatomy, so the region

between them is more likely to be one that real training trajectories actually pass near.

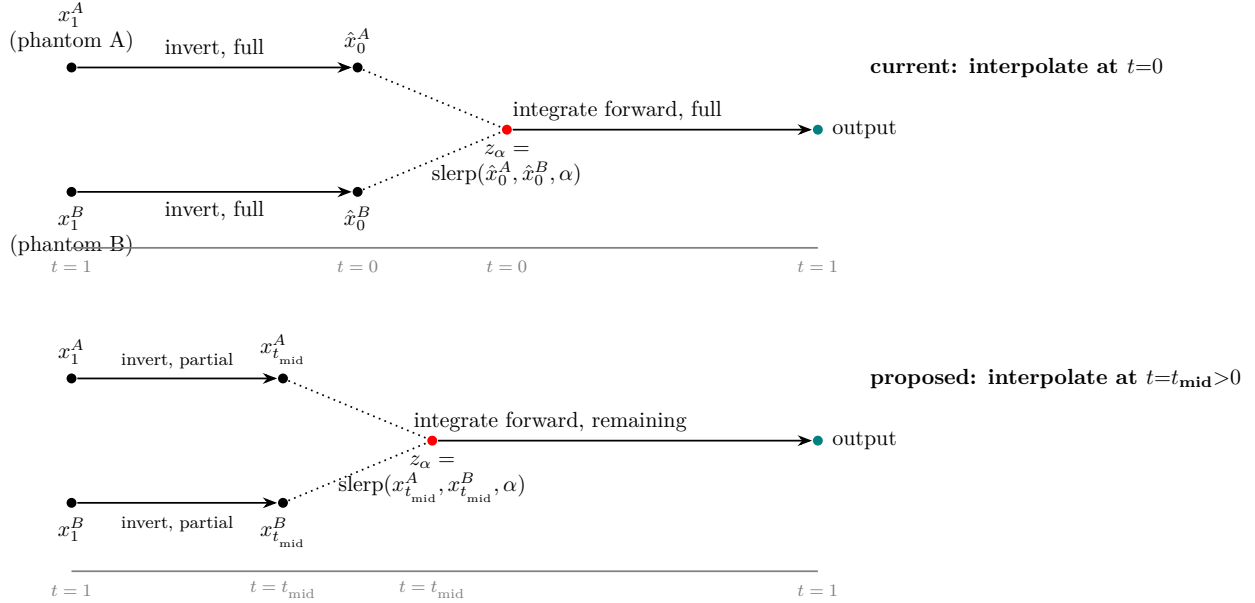


Figure 5: Both rows invert A, invert B, slerp, and integrate forward, the only difference is how far back each phantom is inverted before interpolating. Both A and B are still inverted independently and symmetrically; the proposal only stops the backward integration early.

4.7 Metrics

We use two families of metric, computed directly on the generated label volume. **Centroid MAE** compares, for a pair (A, B) at mixing fraction α , the naively expected lump position $p_{\text{expected}}(\alpha) = (1 - \alpha)p_A + \alpha p_B$ against the actual centroid of the generated lump, averaged over all pair- α combinations. **Component counts** apply 3D connected-component labeling to the generated foreground and lump masks; real single-lump phantoms have exactly one connected component of each, so a value of 1 is the target and larger values indicate torn, fragmented anatomy.

4.8 t_{mid} sweep results and diagnosis

We swept t_{mid} from 0.1 to 0.9 on 20 real-phantom pairs $\times$ 3 mixing fractions (Table 2). Taken at face value this looks like a real improvement: centroid MAE drops by roughly 40% from $t_{\text{mid}} = 0.1$ to its plateau around 0.6–0.7, and anatomy fragmentation drops by more than $4\times$ over the same range.

t_{mid}	centroid MAE (voxels)	avg. anatomy components	avg. lump components
0.1	10.81	22.78	7.18
0.2	7.93	25.82	5.32
0.3	6.86	26.57	2.97
0.4	6.80	23.50	2.23
0.5	6.86	17.67	2.62
0.6	6.55	7.73	3.12
0.7	6.40	5.77	5.15
0.8	7.10	5.07	3.78
0.9	7.15	6.17	1.72

Table 2: t_{mid} sweep, averaged over 20 real-phantom pairs $\times$ 3 mixing fractions.

Inspecting the rendered examples the sweep also logged told a different story than the aggregate numbers. Figure 6 shows the same pair at $t_{\text{mid}} = 0.5$: at $\alpha = 0.25$ the output is essentially identical to phantom A rather than a blend toward B; at $\alpha = 0.5$ it shows *both* phantoms’ lumps simultaneously rather than one lump in between. This pattern held across multiple pairs and across $t_{\text{mid}} \in \{0.1, 0.5, 0.9\}$: the model behaves like a nearest-neighbor selector in angle, reproducing whichever endpoint α is closer to, rather than smoothly interpolating. The centroid and fragmentation gains in Table 2 measure “looks closer to one real endpoint, less torn than a full inversion” rather than “genuinely interpolates between the two.”

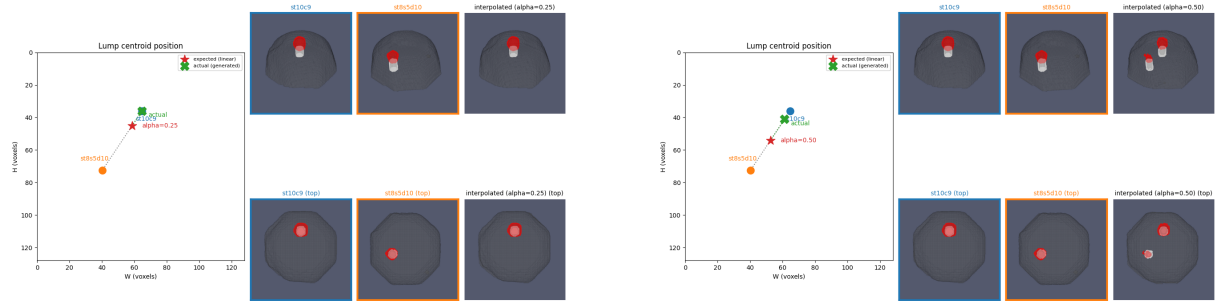


Figure 6: Same pair (st10c9↔st8s5d10), $t_{\text{mid}} = 0.5$. Left, $\alpha = 0.25$: output matches phantom A almost exactly. Right, $\alpha = 0.5$: both A’s and B’s lumps appear simultaneously instead of one blended lump.

4.9 Why interpolation was abandoned

The network was only ever taught the translation from noise-space position to anatomy *at* each individual real phantom’s own inverted point; nothing in training ever supervised the region between two different phantoms’ points, at any t_{mid} . This is a structural consequence of flow matching having no compression bottleneck, not a fixable bug in the interpolation procedure itself, so we set the interpolation goal aside and looked for a task that could give the model an exact, checkable target instead of one judged only by whether it looks anatomically plausible.

5 Part II: A Self-Supervised 180° Rotation Task

5.1 Task definition

Every phantom is augmented with $8 \times$ in-plane rotation at 45° increments, orientation $o \in \{0, \dots, 7\}$. Because $180^\circ = 4 \times 45^\circ$, orientations o and $(o + 4) \bmod 8$ of the same underlying phantom are an exact, bit-for-bit 180° pair, giving a task with a property interpolation never had: an exact, closed-form target for every input,

$$x_0 = \text{rotate}(x_{\text{raw}}, o), \quad x_1 = \text{rotate}(x_{\text{raw}}, (o + 4) \bmod 8). \quad (8)$$

Splitting phantoms by whether the lump sits right or left of the volume’s vertical centerline confirmed the framing geometrically: across all 984 augmented instances the split came out exactly balanced (492/492) with zero ambiguous cases (Figure 7), and every $(o, o+4)$ pair lands on opposite sides by construction. Because x_0 is now always real data rather than Gaussian noise, generation is a single forward integration (Eq. 3) starting from the real x_0 , so there is no inversion step, and no noise-space midpoint to be inaccurate about.

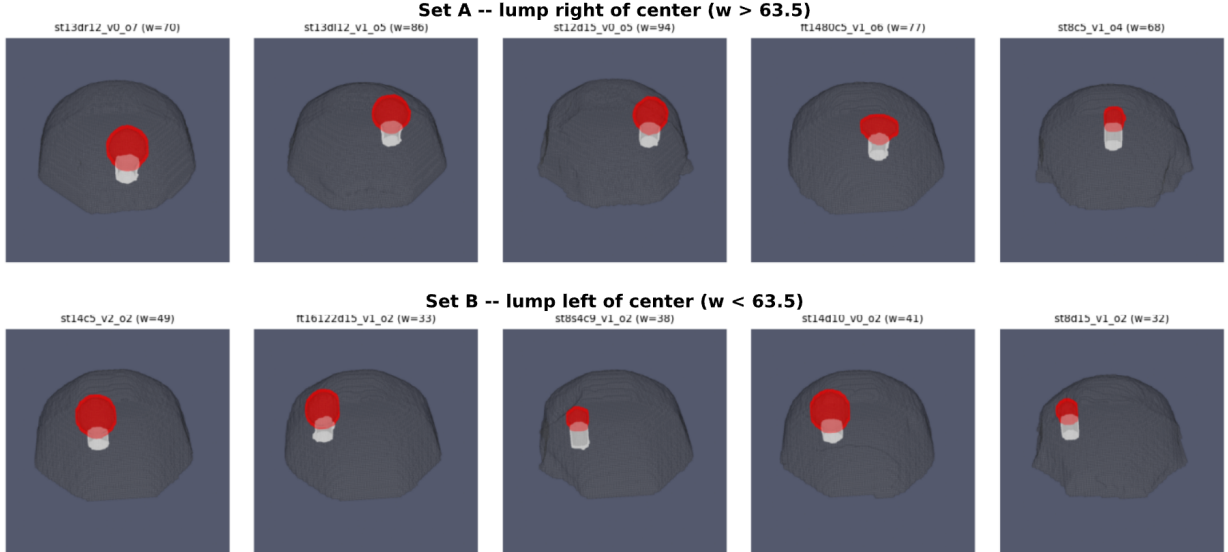


Figure 7: Splitting all 984 augmented instances by lump position ($w \geq 63.5$) gives an exactly balanced 492/492 partition with zero ambiguous cases, confirming that every 180° pair connects opposite sides of the volume.

5.2 Baseline: velocity regression alone

Training with the unweighted loss of Eq. 2 (now with x_0 real rather than noise) produced a striking failure: `train/val_loss` dropped substantially over hundreds of epochs, yet the decoded prediction was voxel-for-voxel identical to x_0 on every tested validation pair, unaffected by using up to $20 \times$ more integration steps at inference. Figure 8 shows a representative example: the predicted lump centroid sits directly on top of the original, far from the true 180° target, yet gross voxel agreement with the target is still 91.6%, since background and insert dominate the volume closely enough between the two orientations that a model which changes nothing at all can still score deceptively well on a coarse, class-blind metric. This observation motivated the class-sensitive, mask-based

metrics used for the rest of this section. Separating the raw velocity prediction’s *direction* from its *magnitude* (checked against the exact training-line target) showed the network already had a consistent directional signal (cosine similarity ≈ 0.65); the predicted velocity magnitude was only about 20% of what was needed to actually flip a voxel’s class after integration, an undercommitted regression target, a typical symptom of plain MSE hedging once loss is already fairly low.

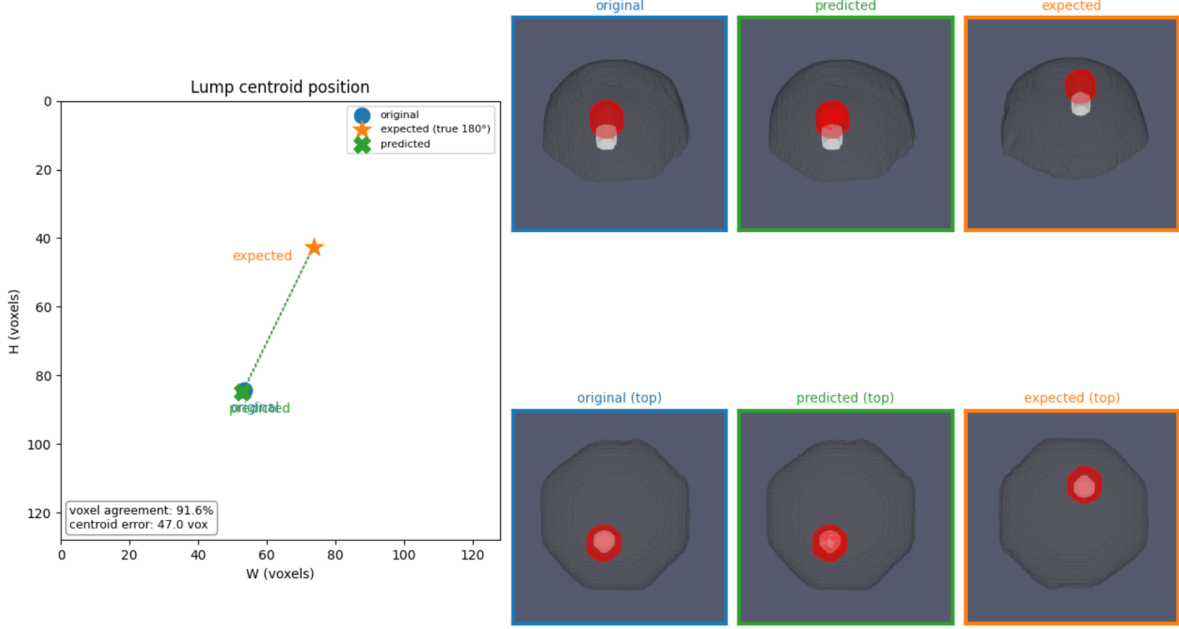


Figure 8: Velocity-MSE-only baseline. The predicted lump position (green) coincides with the original (blue) rather than the true 180° target (orange), despite 91.6% gross voxel agreement with the target.

5.3 Fix 1: a cross-entropy endpoint loss

Since $v^* = x_1 - x_0$ is constant along the training line, algebra gives an endpoint estimate from any point on the trajectory,

$$\hat{x}_1 = x_t + (1 - t) v_\theta(x_t, t), \quad (9)$$

exact when $v_\theta = v^*$. Supervising this estimate with MSE against x_1 turns out to be an exact reweighting of the existing velocity loss ($(\hat{x}_1 - x_1)^2 = (1 - t)^2 (v_\theta - v^*)^2$), not a new training signal, so we used cross-entropy instead, whose gradient does not shrink as a prediction approaches “good enough”:

$$\mathcal{L}_{\text{pixel}} = \text{CE}(\hat{x}_1, \text{argmax}(x_1)), \quad \mathcal{L} = \mathcal{L}_{\text{velocity}} + \lambda_{\text{pixel}} \mathcal{L}_{\text{pixel}}. \quad (10)$$

This directly countered the undercommitment: the decoded prediction started moving for the first time, with a roughly 50–60% reduction in lump-centroid distance within the first ~ 150 epochs. But it introduced a new failure mode: predictions became a massive, fragmented mass covering much of the volume’s cross-section (on the order of 24 disconnected lump components versus about 1 on real phantoms), while centroid distance and gross voxel agreement both stayed misleadingly reasonable, since a large blob’s center of mass can coincidentally sit near the true position. The root cause is specific to how class-weighted cross-entropy is computed: the class weight is applied

by each voxel’s *true* class, not its predicted one, so missing a real lump voxel costs up to $12.57\times$ but wrongly predicting lump at a true-background voxel costs only $1.0\times$, a direct incentive to over-predict the rare classes wherever the network is unsure.

5.4 Fix 2: Dice loss, decaying schedule, and EMA

Three complementary changes addressed the over-prediction. A soft Dice term on the lump/pillar channels penalizes false positives symmetrically with false negatives, unlike cross-entropy:

$$\mathcal{L}_{\text{dice}} = 1 - \frac{2 \sum_v p_v t_v + 1}{\sum_v p_v + \sum_v t_v + 1}, \quad \mathcal{L} += \lambda_{\text{dice}} \mathcal{L}_{\text{dice}}. \quad (11)$$

The cross-entropy weight λ_{pixel} was linearly decayed from 1.0 to 0.3 over training, since held constant it sustained a $\sim 1.25\text{--}1.3\times$ magnitude overshoot for hundreds of epochs. An exponential moving average of the model weights (decay 0.999) was used for every evaluation and checkpoint, motivated by the aggregate lump-centroid error swinging between roughly 17 and 60 voxels from one 25-epoch checkpoint to the next with no clear trend. This combination fixed the gross over-prediction: predicted lump component counts fell from the 20–60 range to mostly 2–14, but, as the next section shows, mask overlap with the true target remained low.

5.5 A trustworthy metric: mask overlap

Directly computing intersection-over-union and Dice between the predicted and true binary lump mask,

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|}, \quad \text{Dice} = \frac{2|A \cap B|}{|A| + |B|}, \quad (12)$$

gave mean IoU ≈ 0.07 and Dice ≈ 0.13 across ten validation pairs after Fix 2, nonzero and therefore meaningful, since the two positions are always 180° apart and can never overlap by chance, but far from good. A small blob that happens to contain one true voxel can score a near-zero IoU while its centroid looks correct, which is exactly why mask overlap (not centroid distance) is used as the trusted metric throughout the remainder of this section.

5.6 Diagnosis: exposure bias, not numerical error

Before revisiting the loss again, we tested whether integration accuracy was the bottleneck: switching the sampler from Euler to Heun’s method, and separately tripling the step count from 30 to 100, both left mean combined IoU essentially unchanged ($0.094 \rightarrow 0.094 \rightarrow 0.090$). This rules out discretization error and points to *exposure bias*: the network is only ever trained at points exactly on the straight line between a real (x_0, x_1) pair, so it fits that narrow, teacher-forced objective well, but at inference it must integrate its own predicted velocity forward, and small errors compound as its self-directed trajectory departs from that line almost immediately.

5.7 Fix 3: rollout-consistency loss

Every loss used so far is computed at a state constructed directly from the known pair (x_0, x_1) , a question with an always-correct answer sitting in the data. At inference, after the first step, the network instead faces a state *it produced itself*, one it never trained on. Rollout loss closes this gap by training on exactly the states the network’s own multi-step process produces: a short, gradient-tracked rollout from x_0 using the model’s own output at each step,

$$x^{(0)} = x_0, \quad x^{(i+1)} = x^{(i)} + v_\theta(x^{(i)}, i\Delta t)\Delta t, \quad i = 0, \dots, k-1, \quad \Delta t = t_{\text{end}}/k, \quad (13)$$

followed by the same endpoint-estimate cross-entropy and Dice terms (Eqs. 10, 11) evaluated at the rolled-out state instead of a point on the true line. This directly penalizes the drift responsible for exposure bias, at the cost of k forward/backward passes per training step with the full computation graph kept live. Applied to a freshly-initialized network this proved unstable (predicted lump components on a from-scratch run grew across the first three epochs rather than settling down); applied as a *continuation* of an already-competent checkpoint it was stable and produced real, sustained improvement.

5.8 Final checkpoint selection and validated results

Every checkpoint saved by the rollout fine-tune was scored under identical conditions on 50 held-out pairs (Table 3). EMA weights beat raw weights at epoch 1250 but *lose* to raw weights at epoch 1500: the smoothing that helps mid-run becomes a liability once the underlying weights have drifted past their best point. More importantly, training longer did not help: the additional 250 epochs made every EMA metric worse, and only partially recovered it for raw weights. Epoch 1250 (EMA) is the best model this run produced, not the final one.

Checkpoint	lump Dice	pillar Dice	combined Dice
epoch 1250 (raw)	0.451	0.153	0.422
epoch 1250 (EMA)	0.498	0.268	0.478
epoch 1500 (raw)	0.470	0.262	0.448
epoch 1500 (EMA)	0.381	0.120	0.354

Table 3: Every checkpoint saved by the rollout fine-tune, scored on the same 50 validation pairs.

Running the full evaluation on epoch 1250 (EMA) across all 176 held-out validation pairs, plus a matched sample of 176 training pairs, gives Table 4. Validation Dice is not lower than train Dice, ruling out overfitting as an explanation for the result. Figure 9 shows the full $t=0 \rightarrow 1$ trajectory for two held-out pairs: the lump visibly separates from its starting position early in the integration and settles close to its true 180° target, with a clean, compact predicted shape rather than the diffuse multi-component blob of Fix 1 or the oversized mass seen at later, overtrained checkpoints.

Split	lump Dice (mean)	pillar Dice (mean)	combined Dice (mean $\pm$ std)
Validation (176 pairs)	0.472	0.267	0.453 ± 0.151
Train (176 pairs)	0.429	0.247	0.405 ± 0.203

Table 4: Final checkpoint (epoch 1250, EMA), full-dataset evaluation.

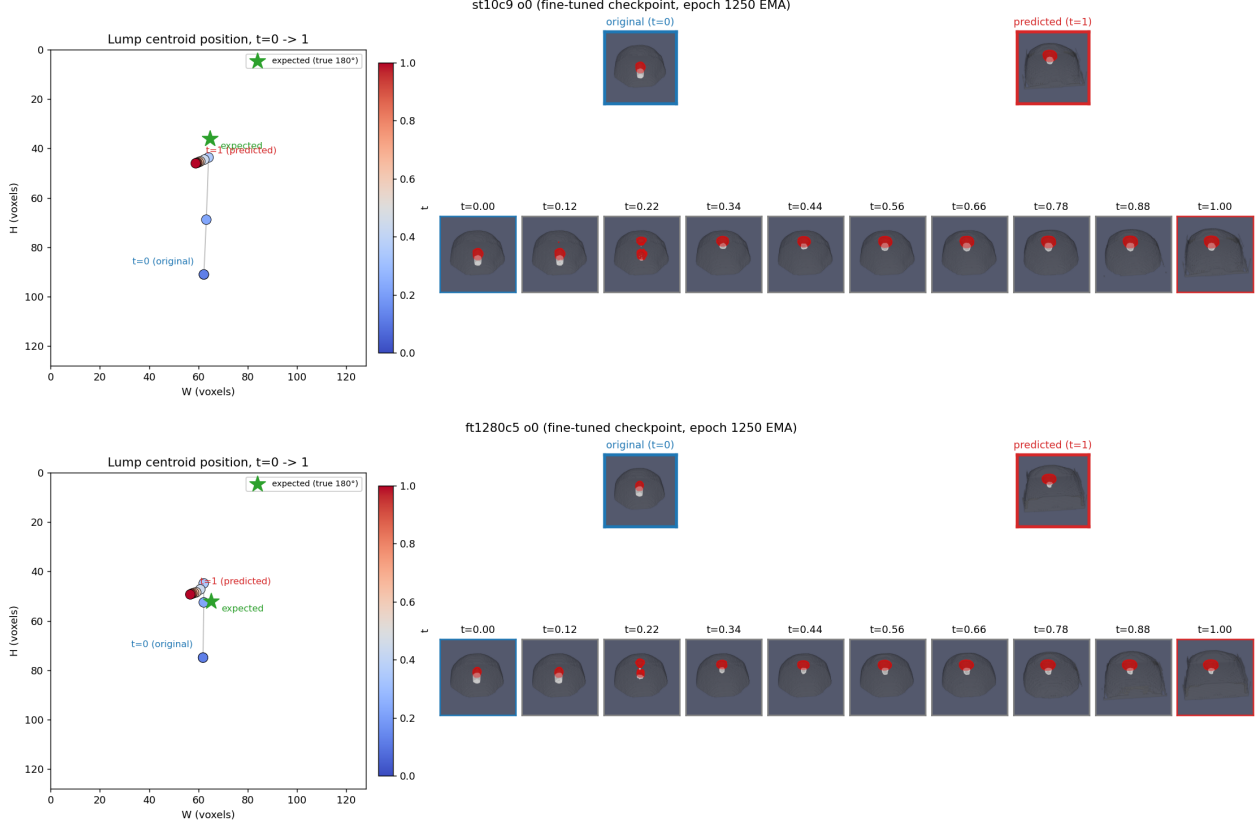


Figure 9: Full $t=0 \rightarrow 1$ trajectory for the final checkpoint on two held-out validation pairs. Left: the lump centroid’s path through the volume, color-graded by t , against the true 180° target (green star). Right: original phantom, intermediate integration steps, and final prediction.

6 Discussion and Conclusion

Table 5 tracks combined mask Dice across the major modeling stages of Part II end to end. The rollout-consistency fix is responsible for roughly a $3.5\times$ improvement in combined Dice over the previous best measured result.

Stage	Comb. Dice	Lump Dice	Notes
Velocity MSE only	n/a	n/a	prediction never moves off x_0
+ cross-entropy endpoint	n/a	n/a	moves, but a $\sim 20\text{--}60$ -component blob
+ Dice loss, decay, EMA	0.13	n/a	blob fixed; still low overlap
+ rollout-consistency fine-tune	0.453	0.472	final model, epoch 1250 EMA, full 176-pair validation

Table 5: Combined mask Dice across the project’s major modeling stages on the rotation task.

Two lessons carried across both parts of this project. First, flow matching’s lack of a compression bottleneck is a genuine trade-off, not a free lunch: it is exactly what gives individual samples and individual-phantom inversion their sharpness, and exactly why the space it learns is never pushed to be smoothly organized as a whole, which is what real phantom-to-phantom interpolation would

have needed. Second, metrics that ignore the underlying class imbalance are actively misleading on this data: a model that changes nothing can still score 91.6% voxel agreement or a deceptively small centroid error, because the classes that matter make up under one percent of every volume; mask overlap on those specific classes was the metric that actually tracked whether the model had learned anything. Once the evaluation caught up to this and the training loss was made to penalize the model’s own multi-step behavior directly, the rotation task went from a model that never moves its prediction at all to one that reliably relocates the lump to its true target, with a validated combined Dice of 0.453 ± 0.151 and no indication of overfitting to the training set.